MSWC English 数据集规模与 8:1:1 划分建议

VE216 Topic 2 语音项目数据说明

基于本地原始数据与当前子集的统计结果

2026 年 4 月 25 日

1 文档目的

这份说明文档回答四个问题：

1. MSWC English 原始数据到底有多大；
2. 当前项目里的 160 词子集为什么会让人感觉“每个辅音的词数不够”；
3. 如果明确要求使用 **8:1:1** 的 train/dev/test 比例，怎样分配更合理；
4. 在当前课程项目规模下，究竟应该取多少词、每词取多少条音频。

本文所有数字都来自本地现有文件：

- `216/project/topic2_speech/data/raw/mswc_en/metadata.json.gz`
- `216/project/topic2_speech/data/raw/mswc_en/en_splits.csv`
- `216/project/topic2_speech/data/raw/mswc_en/audio_en.tar.gz`
- `216/project/topic2_speech/data/raw/mswc_en_subset/selection_summary.json`
- `216/project/topic2_speech/src/prepare_mswc_subset.py`

2 MSWC English 原始数据有多大

2.1 整体规模

表 1 总结了当前本地 **MSWC English** 的规模。

表 1: MSWC English 原始数据整体规模

项目	数值
metadata 中记录的英文词种数	38,174
metadata 中记录的总音频数	6,624,343
`VALID=True` 的有效音频总数	6,621,345
有效 split 覆盖的英文词种数	38,150
train 有效样本数	5,264,225
dev 有效样本数	678,490
test 有效样本数	678,630
`metadata.json.gz` 文件大小	107.75 MB
`en_splits.csv` 文件大小	1.24 GB
`audio_en.tar.gz` 压缩包大小	34.85 GB
本地原始目录总大小	36 GB

这个规模说明：**MSWC English 作为原始数据源完全够大**。当前项目里“某些辅音只有少量单词可用”的问题，不是数据源不够，而是我们后续子集筛选太保守。

2.2 词频分布是明显的长尾分布

MSWC English 不是“每个单词都差不多多”的均匀词表，而是很典型的长尾分布。表 2 给出词频分位数。

表 2: MSWC English 英文词频分位数

10%	25%	50%	75%	90%	95%	99%	最大值
5	7	17	68	238	517	2350	140,555

这组数字的意思是：

- 一半的英文单词，在 metadata 里最多只有 17 条音频；
- 只有头部少量高频词，才能达到几百、几千甚至十几万条；
- 所以如果我们把每词最低门槛设得很高，就会快速丢掉大量中尾部词种。

表 3 给出最常见的 10 个英文词。

表 3: MSWC English 中最常见的 10 个英文词

单词	音频数
was	140,555
and	116,658
you	87,515
that	76,472
are	62,189
his	59,584
this	55,835
for	45,417
have	42,396
what	37,490

因此，设计课程项目子集时，真正要平衡的是两件事：

1. 每个词内部需要保留多少录音；
2. 我们想覆盖多少个不同的词种。

对本题来说，第二件事更关键，因为我们研究的是“某个浊辅音在不同单词中是否有稳定频谱模式”，而不是“同一个单词重复多少次”。

3 当前 160 词子集的问题在哪里

当前项目现成子集来自 `src/prepare_mswc_subset.py` 的默认配置：

- `target-words = 160`
- 每词 `train/dev/test = 60/8/8`
- 总文件数 `160 * (60+8+8) = 12160`

其当前统计见表 4。

表 4: 当前 160 词子集的规模

项目	数值
词数	160
总 WAV 数	12,160
当前每词配额	60 / 8 / 8
train / dev / test 总数	9,600 / 1,280 / 1,280
子集目录大小	390 MB
平均单个 WAV 大小	32,072 Bytes ≈ 31.3 KB
可用浊辅音标签数	15

这个配置有两个明显问题：

1. **比例不是严格的 8:1:1**。 60/8/8 更接近 7.5:1:1。
2. **真正的瓶颈是词种数，而不是录音条数**。每个词拿了 76 条录音，但一共只选 160 个词，所以很多辅音只覆盖到很少的不同单词。

当前子集里，各标签对应的词数如表 5。

表 5: 当前 160 词子集的标签词数分布

标签	词数	标签	词数	标签	词数
b	16	l	32	v	16
d	27	m	28	w	25
dh	14	n	45	y	10
g	11	ng	15	z	15
jh	6	r	29	zh	4

这张表直接解释了“为什么感觉词不够用”：

- `/zh/` 只有 4 个词；
- `/jh/` 只有 6 个词；
- `/y/` 只有 10 个词；
- `/g/` 只有 11 个词。

这时模型很容易学成“这些单词长什么样”，而不是“这个辅音本身长什么样”。

3.1 当前门槛本身就会压缩候选池

当前脚本里，`train/dev/test` 既是最终抽样配额，也是候选词的最低门槛。也就是说，一个词只有在 train 至少 60、dev 至少 8、test 至少 8 时，才有资格进入候选池。

在本地重新统计后：

- 当前 `60/8/8` 门槛下，满足条件的候选词共有 **8,227** 个；
- 如果改成严格 8:1:1 且更温和的 `40/5/5`，候选词会增加到 **10,417** 个。

所以，与其把每个词内部重复录音保得很高，不如把每词配额降一点，再把预算留给更多词种。

4 如果坚持 8:1:1，怎样分配更合理

表 6 比较了几种严格 8:1:1 的每词配额。这里“候选词数”指满足该门槛、能进入候选池的英文词种数；“600 词估算体积”用当前子集平均每个 WAV 的大小近似估算。

表 6: 几种严格 8:1:1 配额方案的比较

每词配额	每词总文件数	候选词数	若选 600 词的总文件数	600 词估算体积
16/2/2	20	16,111	12,000	0.36 GiB
24/3/3	30	13,543	18,000	0.54 GiB
40/5/5	50	10,417	30,000	0.90 GiB
48/6/6	60	9,395	36,000	1.08 GiB
64/8/8	80	7,884	48,000	1.43 GiB
80/10/10	100	6,723	60,000	1.79 GiB

这张表里最重要的结论是：**40/5/5 是最均衡的中间点**。

原因如下：

- 它是严格的 8:1:1；
- 每词共 50 条音频，已经足够做训练、调阈值和最终测试；
- 候选词仍有 10,417 个，不会把词表压得太狠；
- 如果做 600 词子集，总量约 30,000 条、约 0.9 GiB，课程项目可控。

相反：

- `16/2/2` 或 `24/3/3` 太薄，dev/test 容易波动；
- `80/10/10` 太重，磁盘和整理成本上去很快，而且额外预算更多花在“同一单词重复录音”，不如拿去换更多词。

5 我们到底应该取多少词

如果把每词配额固定为 **40/5/5**，只改变总词数，则不同目标规模的含义如表 7。

表 7: 在 40/5/5 下，不同目标词数的含义

目标词数	总文件数	估算体积	代表性覆盖	适用场景
160	8,000	0.24 GiB	太少	不建议，词种仍偏少
300	15,000	0.45 GiB	中等	只做四个浊辅音时可接受
400	20,000	0.60 GiB	中等偏稳	课程交付版比较合适
600	30,000	0.90 GiB	较稳	我最推荐的主实验规模
800	40,000	1.19 GiB	更全面	想覆盖更多浊辅音时可考虑
1000	50,000	1.49 GiB	最全面	成本更高，课程项目未必需要

5.1 为什么我推荐 600 词，而不是继续用 160 词

用当前脚本默认的 greedy 选词逻辑，在 `40/5/5` 下做模拟，几个代表性尾部标签的词数变化如表 8。

表 8: 40/5/5 下，不同目标词数的尾部标签覆盖模拟

目标词数	b	g	jh	y	zh	说明
160	16	11	6	10	4	与当前量级接近，仍然偏小
300	31	25	10	17	4	四标签小实验可用
400	46	29	16	20	4	比 160 明显稳一些
600	65	36	25	29	6	绝大多数标签已明显改善
800	82	51	34	39	8	覆盖进一步提升

这说明：

- 把词数从 160 提到 600，绝大多数标签都会显著改善；
- 对课程项目来说，600 已经是一个很好的中间点；
- 继续增加到 800 或 1000 当然更全面，但收益和成本开始一起上升。

5.2 还要补一个关键点：不能只改词数，还要改选词策略

当前脚本的 `src/prepare_mswc_subset.py` 在选词时使用

```
select_words(candidates, target_words, seed_words_per_label=4)
```

也就是说，每个标签最开始只保底拿 4 个词，然后再做全局 greedy fill。这个策略对头部标签影响不大，但会持续压低尾部标签，尤其是 `/zh/`。

我本地做过一个简单模拟：在 `40/5/5`、`600` 词的设定下：

- 如果继续用当前 `seed_words_per_label = 4`，则 `/zh/` 只有 **6** 个词；
- 如果把每标签保底词数提到 **20**，则 `/zh/` 可以提升到 **20** 个词。

所以真正合理的完整方案不是只改成 8:1:1，而是三件事一起改：

1. 每词比例改成严格 8:1:1；
2. 总词数从 160 提升到 600 左右；
3. 把每标签保底词数从 4 提高到更合理的数，例如 20，或者显式加入 `min_words_per_label` 逻辑。

6 最终建议

把上面的分析压缩成一句话：

MSWC English 原始数据完全够大；当前项目的问题不是数据源太小，而是子集只取了 160 个词。若要改成 8:1:1，我建议优先采用 40/5/5，并把词数提升到 600；如果还想覆盖更多浊辅音，就同步提高每个标签的保底词数。

更具体一点：

表 9: 按任务目标给出的推荐规模

目标	推荐词数	推荐配额	说明
只做四个浊辅音的课程主线	300–400	40/5/5	比当前 160 词稳很多，适合先完成交付
想扩到更多浊辅音	**600**	**40/5/5**	我认为这是最均衡的主方案
想尽量覆盖全部可用浊辅音并做更稳的 AI 比较	800–1000	40/5/5 或 48/6/6	更全面，但成本更高

因此，本文给出的主结论是：

推荐主方案：600 词 + 40/5/5 + 提高每标签保底词数。

附：建议命令

如果后续要落地到新的子集，我建议先以如下命令为目标配置：

```
cd 216/project/topic2_speech
.venv/bin/python src/prepare_mswc_subset.py \
  --target-words 600 \
  --train-per-word 40 \
  --dev-per-word 5 \
  --test-per-word 5
```

这里只差一步：把脚本里的 `seed_words_per_label` 也暴露成命令行参数，便于把默认值从 4 提到 20。这个修改不在本文范围内，但从数据设计角度看，我认为值得做。